



Introduction to C

Key things to note about C

- C is based on a **procedural programming** paradigm, **unlike Java** (which is object oriented)
- Basic building blocks are **functions** and **variables**, **unlike Java** (which uses classes and members)
- Functions and variables in C are **typed**, **as in Java**
- However, C has **weak typing**, meaning that type conversion can be done implicitly, **unlike in Java**
- We will learn what happens behind the scenes during **compilation** and **execution**

Lecture overview

- First program ←
- Basic input and output

Your first C program

```
#include <stdio.h>
```

← use system library functions in order to use printf/scanf

```
int main() {
```

```
    printf ("Hello, world!\n");
```

← explicitly specify newline

```
    return 0;
```

← functions should specify return value

```
}
```

/share/classes/class04/hello.c

- Every C program should have exactly one function named **main** (and possibly other functions, as we'll see in upcoming lectures)
- Function **main** should return an integer value – the exit status of our program
- Return value 0 for **main** typically indicates fine execution
- Return value of **main** can be retrieved by typing **echo \$?** in the command line after execution.

Compiling your C program

- To compile C source code into an executable binary, you should use a C compiler, such as *gcc* or *cc*

```
gcc hello.c
```

- This will create an executable file called '**a.out**', which can then be run:

```
./a.out
```

```
Hello, world!
```

- Compilation is system-dependent. **a.out** is only guaranteed to run appropriately on the machine it was compiled on

Compiling your C program

- You can specify the name of the generated executable using the compiler's '-o' option:

```
gcc -o hello hello.c
```

By convention, the executable typically has the same name as the C file, but without the '.c' extension

- This can run as:

```
./hello
```

```
Hello, world!
```

Compilation errors / warnings

- If file cannot be compiled due to syntax errors, error messages will be issued
- Warnings will still allow compilation. But they could indicate hidden bugs and semantic errors
- We will compile our programs with the '**-Wall**' option to generate a base set of warnings (with '**-Wall**', there are even stricter configurations):
 - e.g., warning that a declared variable is unused, warning that a variable is used before being initialized

```
gcc -Wall -o hello hello.c
```

Lecture overview

- First program
- Basic input and output ←

Input and output

- C programs have two forms of output:
 - Exit status (integer return value of main)
 - Output printed to file / output stream
- C programs have two forms of input:
 - Command line arguments (list of strings)
 - Input read from file / `stdin`
- Today we will see output / input using `stdout` / `stdin`
 - The function `printf()` prints to `stdout`
 - The function `scanf()` reads from `stdin`
 - Include `stdio.h` to use either of them

Output - the `printf` function

- The first argument is the control string
- The control string contains text and **conversion specifiers** (indicated by %) for injecting variable content
- For each conversion specifier in the control string, there is an additional argument (in order)

```
int i = 5;
printf ("Arithmetic operation:\n");
printf ("%d + %d = ", 3, i);
printf ("%d\n", 3+i);
```

← any expression can be used

- `printf` has a variable number of parameters (uncommon in C)
- `printf` returns the length of the string it printed (including newline)

Conversion specifiers

<u>Specifier</u>	<u>Corresponding argument</u>
<code>%c</code>	Character
<code>%d</code>	Decimal integer
<code>%e</code>	Floating point – scientific notation
<code>%f</code>	Floating point
<code>%lf</code>	Long float (double)
<code>%s</code>	String

Defining field width

- With `printf()`, it is possible to define the field width for each of the printed arguments
- This is done by adding a number before the conversion specification. The value is aligned to the right and padded on the left with spaces

```
printf ("Twelve:%6d\n", 12);  
printf ("Chars:%3c%3c%3c\n", 'a', 'b', 'c') ;  
printf ("Two thirds:%6.3f\n", (2.0 / 3));
```

```
Twelve:  12  
Chars: a b c  
Two thirds: 0.667
```

Defining field width

- Default is that the value is aligned to the right and padded to the left with spaces.
 - use '-' before the field width for left aligned printing
 - use '0' to pad the left with zeros

```
int a = 5, b = 12;  
  
printf ("%d,%d\n", a, b);  
printf ("%4d,%4d\n", a, b);  
printf ("% -4d,% -4d\n", a, b);  
printf ("%04d,%04d\n", a, b);
```

```
5,12  
  5, 12  
5  ,12  
0005,0012
```

Input - the `scanf` function

- The function `scanf()` reads from `stdin` and, like `printf()`, its first argument is a control string with conversion specifiers

```
scanf ("%d", &x);
```

- The symbol `'&'` is the address operator – the value read is written in the memory space of variable `x`
- The `'%d'` tells `scanf()` to try to read an integer from the standard input
- `scanf` has a variable number of parameters (uncommon in C)
- `scanf` returns the number of values successfully read as input (should be number of arguments - 1)

Input and output – example 1

```
#include <stdio.h>

int main() {

    int n1, n2; ← Variables should be defined at top of function
                  (unenforced convention)

    printf ("Input two integer numbers:");
    scanf ("%d%d", &n1, &n2);
    printf ("\nThe sum of the two numbers is:\n");
    printf ("%d+%d=%d\n", n1,n2,n1+n2);
    return 0;
}
```

`/share/classes/class04/sum.c`

Input and output – example 1

```
#include <stdio.h>

int main() {

    int n1, n2, res;

    printf ("Input two integer numbers:");
    res=scanf ("%d%d", &n1, &n2);
    if(res<2) {
        printf("Wrong input. Expecting two integers.\n");
        return 1; ← Exit status ≠ 0 marks error
    }
    printf ("\nThe sum of the two numbers is:\n");
    printf ("%d+%d=%d\n", n1,n2,n1+n2);
    return 0;
}
```

/share/classes/class04/sum-with-check.c

Input and output – example 2

```
#include <stdio.h>

int main() {

    char    c1, c2;

    printf ("Input two characters:");
    scanf ("%c%c", &c1, &c2);
    printf ("\nThe data that you typed in:\n");
    printf ("%3c%3c\n", c1, c2);
    return 0;
}
```

- For **characters** – **scanf ()** reads in sequence
- Other types – **scanf ()** reads with white space separators
- Better to use file parsing functions, which we see later in the course

Getting help for C functions

- To get the manual page for a C system library function (e.g., `printf`) write:

```
man 3 printf
```

- The '3' tells `man` to look in the 3rd section of the manual pages, which contains help on C library functions
- These pages can also be found online (e.g., http://man7.org/linux/man-pages/dir_section_3.html)

At home...

- Homework exercise #3 due **Sunday, May 11 @ 21:00**
- Before lecture 5, familiarize yourself with **control flow in C**.
 - Guide on Piazza, in Resources section: *Sysprog-guide-C-control-flow.pdf*.
 - We highlighted the main differences from Java
 - We do not plan to review this in class, but we'll clarify any questions you may have
- Before lecture 5, familiarize yourself with **binary representation** of numbers and transformation between decimal and binary bases.
 - Guide on Piazza, in Resources section: *Sysprog-basics-about-binary.pdf*